

1 Introduction

Doc. v1.0

Dans ce projet nous allons implémenter une méthode de reconstruction de réseau de régulation génétique appelé Weighted Genetic Network Analysis (WGCNA) [1, 2].

Soumission Le projet se fait par groupe de deux personnes maximum et peut être réalisé dans n'importe quel langage mais je vous conseille python. Le rendu se fait par mail à pierre.parutto@epita.fr sous la forme d'une archive nommée (dans le cas d'un projet solo, un seul nom est présent) :

`projet_binf_nom1[_nom2].tar.bz2`.

Par exemple : `projet_binf_dupont_parutto.tar.bz2` ou `projet_binf_parutto.tar.bz2` si je suis solo. Le format de l'archive doit être le suivant :

```
$ tar -tf projet_binf_nom1[_nom2].tar.bz2
projet_binf_nom1[_nom2]/
projet_binf_nom1[_nom2]/README
projet_binf_nom1[_nom2]/AUTHORS
projet_binf_nom1[_nom2]/src/
projet_binf_nom1[_nom2]/src/[fichiers de code]
```

où le fichier *AUTHORS* contient les prénoms et noms des membres du groupe et le fichier *README* indique comment utiliser (et compiler selon le langage) votre projet et doit lister **toutes les dépendances** à requises. Votre dépôt peut aussi contenir d'autres fichiers / dossiers nécessaires pour votre code. **Une pénalité d'un point pourra être appliquée en cas de rendu ne respectant pas le format ci-dessus.**

Évaluation 1. Le projet sera évalué automatiquement via une moulinette ; 2. Votre programme sera testé uniquement avec des valeurs d'entrées valides.

Finalement n'hésitez pas à poser vos questions sur discord ou par mail.

2 Palier 1 : préparation des données (3 points)

Programme `clean_dataset` (avec possiblement une extension, ie. *.py*, *.jar*, ...)

Paramètres d'entrée `datadir` `dataname` `K`, où `datadir` est le chemin vers le dossier contenant les données, `dataname` le nom du fichier *csv* contenant les données brutes et `K` est le nombre de gènes à conserver.

Format de sortie Vous devez créer un fichier au format *csv* nommé `[dataname]_cleaned_[K].csv` (où `[dataname]` et `[K]` sont à remplacer par leurs valeur de paramètres d'entrées associés) situé dans le dossier donné par le paramètre `datadir`. Ce fichier a le même format que le fichier d'entrée mais avec les valeurs arrondies à 5 décimales.

Format d'entrée Le fichier *csv* d'entrées est au format suivant : la première ligne contient les identifiants de chaque colonne (la première colonne de cette ligne est vide), ensuite pour chaque ligne, la première colonne contient son identifiant et les colonnes suivantes une valeur d'expression pour chaque gène. Les données d'expression génétiques sont structurées sous la forme d'une matrice D de taille $M \times N$ où M est le nombre d'observations (échantillons, cellules ou patients) et N le nombre de gènes mesurés.

Principe : A ce palier, on veut enlever les gènes (colonnes de D) qui ne fluctuent pas beaucoup entre observations. Pour ce faire : calculer la variance de chaque gène et garder uniquement les N gènes avec la plus forte variance.

3 Palier 2 : construction du réseau (7 points)

Programme `build_graph` (avec possiblement une extension, ie. *.py*, *.jar*, ...)

Paramètres d'entrée `datadir` `dataname` `option` [`beta`], où `datadir` est le chemin vers le dossier contenant les données, `dataname` le nom du fichier `csv` contenant les données (au format du palier précédent). Les options possibles sont :

- `POW` : le programme ne prend pas d'autre paramètre et affiche sur la sortie standard valeur optimale de β calculée (cf ci-dessous).
- `TOM` : le programme utilise aussi le paramètre `beta` et calcule la matrice `TOM` basée sur celui-ci (cf. ci-dessous).

Format de sortie Pour l'option `TOM` uniquement : un fichier au format `csv` nommé `[dataname]_TOM_[beta].csv` (où `[dataname]` et `[beta]` sont à remplacer par la valeur des paramètres d'entrées associés) situé dans le dossier donné par le paramètre `datadir`. Ce fichier contient la matrice de recouvrement (`TOM`, définie ci-dessous), les valeurs d'une même ligne sont séparées par des virgules et arrondies à 5 décimales. **Note** : à cette étape, on conserve uniquement les données de la matrice, pas les noms des lignes et colonnes.

Étape 1 : Matrice de corrélation Pour construire le réseau d'expression génétique, on va calculer la corrélation entre l'expression de chaque gène dans les observations. Pour une matrice de données D et deux gènes d'indices i et j , la corrélation est donnée par :

$$corr_{i,j} = \frac{Cov_{D_{*,i}, D_{*,j}}}{\sqrt{Var[D_{*,i}] \times Var[D_{*,j}]}}$$

avec Cov la covariance, Var la variance et le symbole $*$ en indice indique que l'on considère toute la ligne / colonne. Ici vous pouvez utiliser la fonction `corrcoef` de `numpy`. On transforme ensuite cette corrélation dans $[-1, 1]$ en une valeur dans $[0, 1]$ que l'on conserve dans une matrice A de taille $N \times N$ où N est le nombre de gènes (colonnes) de D :

$$a_{i,j} = \frac{corr_{i,j} + 1}{2}.$$

Étape 2 : Détermination du seuil mou β Plutôt que de binariser la matrice de corrélation, on va utiliser un seuil "mou" β et définir une nouvelle matrice α :

$$\alpha_{i,j} = (a_{i,j})^\beta.$$

Pour déterminer β , on fait l'hypothèse que notre réseau doit être "scale-free", une propriété observée dans la plupart des réseaux biologiques. Dans un tel réseau, la distribution des poids de connectivité k suit une loi de puissance :

$$p(k) \sim k^{-\beta}$$

où p représente la densité de probabilité et β est le paramètre de la distribution. On va alors chercher la valeur positive entière minimum de β tel que la densité de probabilité observée $\tilde{p}$ soit la plus proche d'une loi de puissance. Pour ce faire, voici les étapes à suivre :

1. Calculer pour chaque gène sa connectivité c :

$$c_i = \sum_{j=0}^{N-1} \alpha_{i,j}$$

2. Générer la distribution empirique $\tilde{p}$ via un histogramme, vous pouvez utiliser `numpy.histogram`. Récupérer les coordonnées b des paquets (bins) et les valeurs o associées. Utiliser `ceil( $\sqrt{N}$ )` paquets, où N est le nombre de gènes et `ceil` la fonction arrondissant à l'entier supérieur.
3. Si besoin, convertissez les valeurs des paquets b_i pour qu'elles représentent les centres et non les bords de chaque paquet.
4. Enlever toutes les paires (b_i, o_i) , $0 \leq i < N$, pour lesquelles $o_i = 0$.

5. Une loi de puissance implique une relation linéaire en axes log-log. On va donc approximer notre distribution par la droite : $\log_{10}(o) = u \times \log_{10}(b) + v$, en utilisant, par exemple, `linregress` de `scipy.stats`.
6. Les coefficients (u, v) en eux même ne nous intéressent pas, ce qu'on veut c'est récupérer le coefficient de détermination associé à cette valeur de β : R_β^2 .
7. Répéter ces étapes pour toutes les valeurs entières $0 < \beta \leq 30$ et retourner la plus petite valeur de β tel que $R_\beta^2 > 0.8$.

Étape 3 : Détermination de la matrice de connectivité Finalement, la matrice de connectivité finale du graphe est donnée par la matrice de topologie recouvrante (Topological Overlap Matrix, TOM) T définie par :

$$t_{i,j} = \frac{|\alpha_{i,j} + \sum_{u \neq i,j} \tilde{\alpha}_{i,u} \times \tilde{\alpha}_{u,j}|}{\min(c_i, c_j) + 1 - |\alpha_{i,j}|},$$

avec $|\cdot|$ la valeur absolue, $\tilde{\alpha}_{i,j} = \alpha_{i,j} \times \text{sign}(\text{corr}_{i,j})$ et sign la fonction donnant le signe.

4 Palier 3 : Construction des modules génétiques (7 points)

Programme `find_modules` (avec possiblement une extension, ie. `.py`, `.jar`, ...)

Paramètres d'entrée `datadir` `dataname` `option` `tau` [1], où `datadir` est le chemin vers le dossier contenant les données, `dataname` le nom du fichier `csv` contenant les données (au format du palier précédent), `option` peut être soit `static`, `adaptive` ou `dynamic`, avec :

- Pour l'option `static`, on donne le paramètre `tau` (type `int`) correspondant au seuil de significativité des points de cassures et le paramètre `l` (type `float`) correspondant au seuil de découpe souhaité et .
- Pour l'option `dynamic`, on donne le paramètre `tau` (type `int`) correspondant au seuil de significativité des points de cassures.
- Pour l'option `adaptive` on donne le paramètre `tau` (type `int`) correspondant au seuil de significativité des points de cassures.

Format de sortie Un fichier texte nommé `[dataname]_[option]_[1]_[tau].csv` dans le cas statique ou `dataname_[option]_[tau].csv` dans les autres cas (les éléments entre crochets sont à remplacés par les valeurs des paramètres associés). Ce fichier contient sur chaque ligne k l'id du groupe auquel appartient le k ème gène de la matrice TOM . L'ordre doit être le même que celui des lignes et colonnes de la matrice d'entrée.

Principe Maintenant que l'on a un graphe, sous forme de matrice de connectivité, on va chercher les ensembles de gènes très corrélés entre eux. Cela va nous permettre de construire des "modules" de gènes se comportant de manière similaire [3].

Étape 1 : construction de l'arbre initial On commence par calculer un arbre de distances entre gènes appelé dendrogramme. Vous pouvez utiliser la fonction `linkage` de `scipy.hierarchy` en utilisant les options `method="average"` et `optimal_ordering=True` sur $1 - T$ où T est la matrice de topologie recouvrante obtenue à la fin du palier précédent (**Attention** : le format d'entrée de `linkage` est une matrice sous forme compressée). Pour obtenir un arbre T à partir de la matrice de liaisons on peut utiliser la fonction `to_tree` de `scipy.cluster.hierarchy`. Une fois qu'on a cet arbre, on veut obtenir la liste des distances associées à chaque noeud interne dans l'ordre de T (c'est à dire un parcours profondeur main gauche infixe). On appelle cette liste la liste des hauteurs h .

Étape 2 : Découpe statique de l'arbre L'arbre généré par `linkage` donne une hiérarchie de distances entre noeuds mais ne fourni pas de groupes. Pour les obtenir, on va découper l'arbre à un certain niveau de hauteur. A partir du tableau des hauteurs h et d'un niveau de hauteur l , on défini le tableau $\bar{h} = h - l$, dans ce tableau, on cherche les points transitions correspondant à l'ensemble des positions k tel que $\bar{h}_k > 0$ et $\bar{h}_{k+1} < 0$. A chaque transition, on associe deux éléments (cf. Figure 1) :

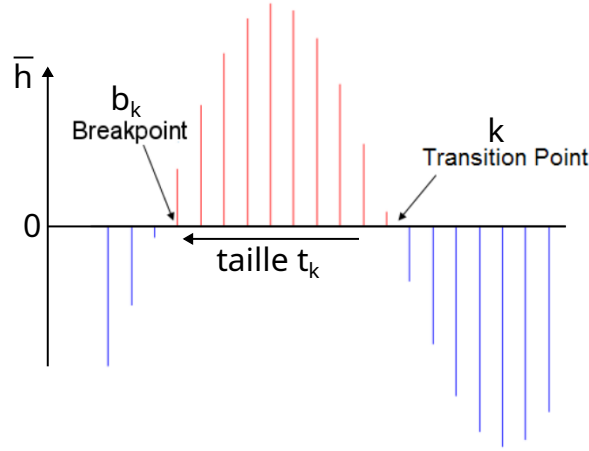


FIGURE 1 – Visualisation d'un point de transition, de sa taille et son point de cassure associé à partir d'un tableau $\bar{h}$

- sa taille t_k qui correspond au nombre de positions successives $< k$ de même signe que $\bar{h}_k$.
- son point de cassure (*breakpoint*) $b_k = k - t_k$, la première position où la valeur de $\bar{h}$ devient du même signe que $\bar{h}_k$. Si on atteint le début de la liste sans trouver une telle position, alors on choisira $b_k = 0$.

Pour trouver les groupes, on va chercher tous les points de cassures dans $\bar{h}$, puis garder uniquement les points significatifs, dont la taille de la transition associée est $> \tau$. Tous les éléments entre deux points de cassures significatifs font parti du même groupe. **Attention :** Pour éviter de perdre des éléments, on va toujours considérer que la position 0 est le premier point de cassure significatif et $|\bar{h}| - 1$ le dernier.

Étape 3 : Découpe adaptative La méthode de découpe statique n'indique pas comment trouver le seuil de découpe l . Pour éviter les problèmes liés au choix d'un seuil arbitraire, on va calculer des valeurs de seuils adaptées pour chaque groupe. Soit un groupe H composées de n distances $h_1, \dots, h_n$ alors on a :

$$\begin{aligned} l_m &= \frac{1}{n} \sum_{i=1}^n h_i \\ l_u &= \frac{1}{2} [l_m + \max(h_1, \dots, h_n)] \\ l_d &= \frac{1}{2} [l_m + \min(h_1, \dots, h_n)] \end{aligned}$$

A partir de ces quantités, on définit la procédure de découpe adaptative pour un groupe de hauteurs $H = h_1, \dots, h_n$:

1. Calculer l_m, l_d, l_u
2. G = Découpe statique de H avec le seuil l_m
3. $G = G'$
4. Si $|G| == 1$ alors $G' =$ Découpe statique de H avec le seuil l_d
5. Si $G == G'$ alors $G' =$ Découpe statique de H avec le seuil l_u
6. retourner G'

où $|G|$ représente le nombre de groupes différents dans G .

Étape 4 : Découpe dynamique de l'arbre Finalement, à partir des deux procédures précédentes on peut définir une procédure de découpe dynamique d'un tableau de hauteurs $H = h_1, \dots, h_n$:

1. Déterminer un premier ensemble de groupes $\Omega = H^{(1)}, \dots, H^{(K)}$ avec une découpe statique utilisant une haute valeur de seuil $l = 0.99 \times \max(H)$.
2. Pour chaque groupe $H^{(k)} \in \Omega$: Appeler récursivement la procédure de découpe adaptative (étape 3) sur $H^{(k)}$ et tous les sous groupes ainsi générés jusqu'à convergence, quand plus aucun nouveau groupe n'est formé.

Finalement, quelque soit la méthode, une fois le tableau des groupes obtenu il faut propager les groupes aux feuilles de l'arbre. Le groupe d'une feuille correspondant au groupe de son noeud parent.

Attention : l'ordre dans le tableau final doit être le même que celui de la matrice d'entrée.

5 Palier 4 : Extraction des eigengenes (3 points)

Programme `extract_eigen` (avec possiblement une extension, ie. `.py`, `.jar`, ...)

Paramètres d'entrée `datadir` `dataname` `groupname`, où `datadir` est le chemin vers le dossier contenant les données, `dataname` le nom du fichier `csv` contenant les données d'expression génétique (du palier 1) et `groupname` le nom du fichier `csv` contenant les informations de groupes pour chaque gène (au format du palier précédent).

Format de sortie Un fichier `csv` nommé `[grpname]_eigen.csv` où chaque ligne correspond à un groupe, trié par ordre d'identifiants, et les colonnes sont les valeurs du vecteur d'eigengene (arrondies à 3 décimales) séparées par des virgules. Pour chaque ligne, la première valeur est l'id du groupe. Notez que le nombre de valeurs va varier entre chaque groupe.

Principe Pour avoir une représentation génétique de chaque groupe, on va calculer son *eigengene*. Pour un groupe G :

1. Extraire l'ensemble K des positions des gènes présents dans G .
2. Récupérer la sous-matrice d'expression de ces gènes $D^{(G)} = D_{*,K}$.
3. Centrer et normaliser $D^{(G)}$:

$$D^{(G)} = \frac{D^{(G)} - E[D^{(G)}]}{\text{Var}[D^{(G)}]},$$

où E et Var sont respectivement la moyenne et variance calculées sur tous les éléments de la matrice.

4. Calculer la décomposition en valeurs singulières (SVD) de $D^{(G)}$:

$$D^{(G)} = U \times X \times V^T,$$

où U , X et V sont trois matrices. Vous pouvez utiliser la fonction `svd` de `numpy.linalg`.

5. L'eigengene du groupe G correspond à la première colonne de la matrice V (**Attention** : on parle ici de V et pas V^T).

6 Exemples

Vous trouverez des exemples d'exécution de chaque paliers [ici](#) et les pdfs des articles cités [ici](#). Les commandes utilisées et sorties associées sont présentées dans le fichier `test_[date]` dans le dossier de tests.

Références

- [1] S. Morabito, F. Reese, N. Rahimzadeh, E. Miyoshi, and V. Swarup, “hdwgcna identifies co-expression networks in high-dimensional transcriptomics data,” *Cell reports methods*, vol. 3, no. 6, 2023.
- [2] B. Zhang, S. Horvath, *et al.*, “A general framework for weighted gene co-expression network analysis,” *Statistical applications in genetics and molecular biology*, vol. 4, no. 1, p. 1128, 2005.
- [3] P. Langfelder, B. Zhang, and S. Horvath, “Defining clusters from a hierarchical cluster tree : the dynamic tree cut package for r,” *Bioinformatics*, vol. 24, no. 5, pp. 719–720, 2008.